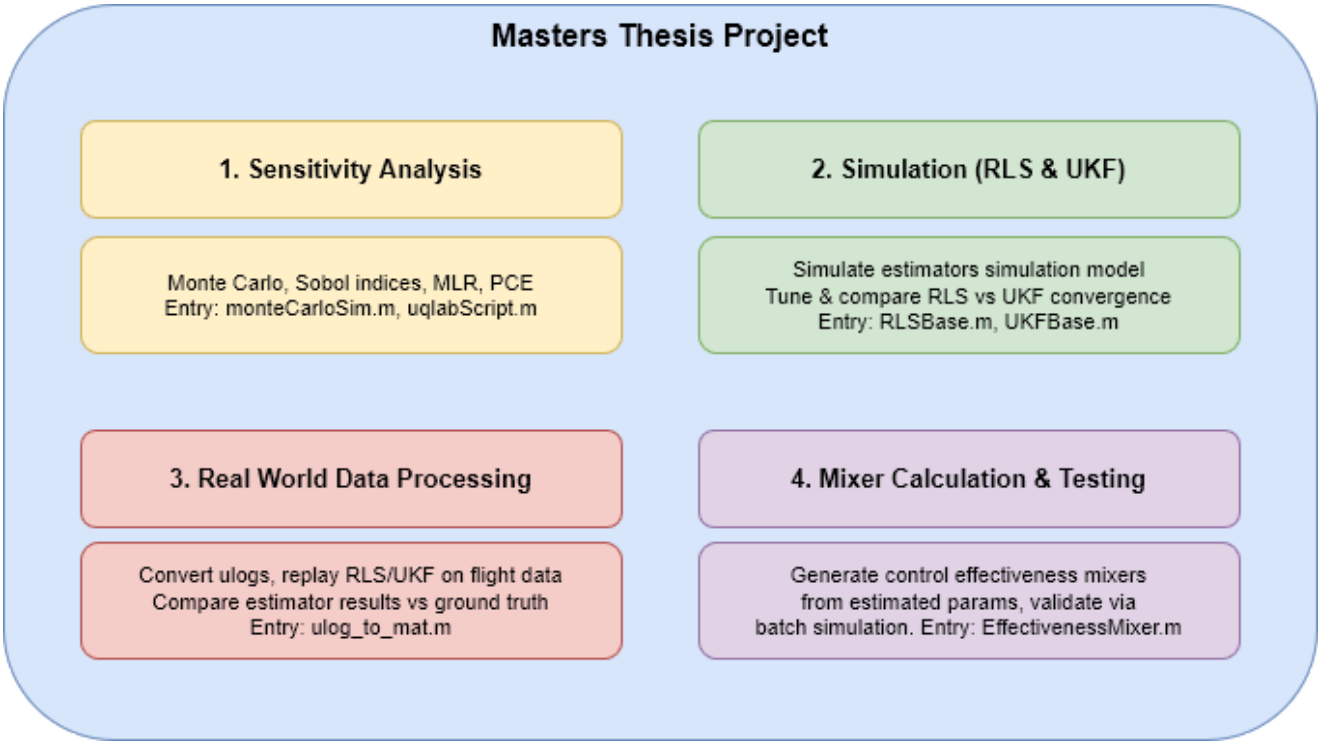
Parameter Estimation for Control Allocation JS-ME UAV Parameter Estimation — Masters Thesis

> **Platform:** Octocopter (Flamingo — Canted Stacked) + Hexacopter variants
> **Flight Controller:** PX4 v1.8
> **Language:** MATLAB / Simulink
> **Core Research:** Online identification of UAV inertial & aerodynamic parameters, sensitivity analysis, and mixer recalculation from estimated parameters.

Four Main Research Sections



Section Index

#	Section	Key Folder(s)	Entry Point(s)
1	01_Sensitivity_Analysis	`Parameter Sensitivity/`	`monteCarloSim.m`, `uqlabScript.m`
2	02_Simulation_RLS_UKF	`RLS/`, `UKF/`, `DataProcessing/`	`RLS/RLSBase.m`, `UKF/UKFBase.m`
3	03_RealWorld_Processing	`EXP_Processing/`, `Flamingo Data/`, `ConvertedUlogs/`	`EXP_Processing/ulog_to_mat.m`

#	Section	Key Folder(s)	Entry Point(s)
4	04_Mixer_Testing	`MixerTesting/`	`EffectivenessMixer.m`, `MixerValidationBase.m`

Shared Infrastructure

> These scripts and data files are not owned by any single section — they are the common foundation that all four sections build on. Understanding these first makes the rest of the codebase much easier to navigate.

UAV Configuration Files

These `.mat` files define the physical configuration of each UAV platform.
Every simulation and analysis section loads one of these as its starting point.

File	Platform	Notes
`Octo_CantedStacked_Flamingo.mat`	Flamingo octocopter	Primary platform — current active config
`Octo_PHex.mat`	PHex hexacopter	Planar hex variant
`Hex.mat`	Generic hexacopter	Used by `HEXTest/` module
`windData.mat`	N/A	Wind disturbance profile used across simulations

Global Parameters

Script	Role
`parameters.m`	Defines all UAV physical constants, simulation settings and estimator hyperparameters in one place — the single source of truth for parameter values across the whole project

---

Parameter Set Generation

Parameter sets are the mechanism that makes Monte Carlo and batch runs repeatable — generate once, reuse across all sections.

Script	Role	Output
`GenerateParameterSet.m`	Samples varied UAV parameter combinations (mass, inertia, COM, motor props) from distributions defined in `parameters.m` and saves each as a numbered `.mat` file	`ParameterSet/ParamSet_001.mat` … `ParamSet_050.mat`

Saved Parameter Set Folders

Folder	Platform	Samples	Purpose
`ParameterSet/`	Flamingo	50	Main batch — used by Sections 2, 3 & 4
`ParameterSetCOMZ/`	Flamingo	—	COM Z-axis variation study
`ParameterSetHEX/`	Hexacopter	1	Hex platform batch
`ParameterSetPHEX/`	PHex	1	PHex platform batch

Trajectory Generation

Used by both simulation (Section 2) and mixer validation (Section 4) to produce reference trajectories and excitation inputs.

Script	Role	Notes
`generate_trajectory.m`	Generates smooth reference position/attitude trajectories for simulation	General purpose hover-and-move profiles
`generate_6dof_step_trajectory.m`	Generates 6-DOF step inputs across all axes — used for estimator excitation and mixer step response testing	Primary excitation signal for RLS/UKF

Simulation Runners

Top-level scripts that orchestrate how simulations are executed. Used by Sections 2 and 4.

Script	Role	Notes
`RunSingle.m`	Runs one simulation with one parameter set — primary tool for development, tuning and debugging	Use this first when investigating a specific case
`RunVariedLoop.m`	Iterates over all parameter sets in `ParameterSet/` sequentially	Use for smaller batches or when parallelism isn't available
`RunVariedLoopParallel.m`	Same as `RunVariedLoop.m` but parallelised with `parfor`	Use for full 50-sample Monte Carlo runs

---

Estimation Core

Script	Role
`ParameterEstimationBase.m`	Parent estimation loop — defines the shared structure that `RLSBase.m`, `UKFBase.m` and their EXP counterparts all build on
`ParameterEstimationPostProcessing.m`	Top-level results aggregator — combines outputs from RLS and UKF across both simulation and real-world runs for direct comparison
`evaluateUAVPerformance.m`	Computes standardised performance metrics (tracking error, settling time, parameter convergence) — called by post-processing scripts across all sections

---

Simulink Bus Definitions

Required for any script that interfaces with the Simulink simulation models.

Script	Role
`MLEBus.m`	Defines the Simulink bus object for MLE (Maximum Likelihood Estimation) signals — must be run before opening any Simulink model that uses this bus
`mlebusgen.m`	Generator script that creates and saves `MLEBus.m` — run this if the bus definition needs to be regenerated
`MLE.m`	Maximum Likelihood Estimation helper — used alongside RLS/UKF for parameter uncertainty quantification

---

Debug Utilities

Small helper scripts used during development across all sections.

Script	Role
`dispStruct.m`	Recursively pretty-prints a MATLAB struct to the console — useful for inspecting nested parameter/result structures
`sizeCheck.m`	Validates matrix and signal dimensions match expected sizes — used to catch dimension mismatches in estimator signals early

---

Shared Infrastructure

These files are used across multiple sections:

File	Purpose
`parameters.m`	Global UAV parameter definitions
`GenerateParameterSet.m`	Samples parameter sets for Monte Carlo / batch runs
`generate_trajectory.m`	Generates reference trajectories for simulation
`generate_6dof_step_trajectory.m`	6-DOF step inputs for excitation
`ParameterEstimationBase.m`	Core estimation loop (parent to RLS/UKF base scripts)
`ParameterEstimationPostProcessing.m`	Aggregates results across estimation runs
`RunSingle.m` / `RunVariedLoop.m`	Single and batch simulation runners
`RunVariedLoopParallel.m`	Parallelised batch runner

File	Purpose
`evaluateUAVPerformance.m`	Scores estimation quality metrics
`dispStruct.m`	Pretty-prints MATLAB structs (debug utility)
`sizeCheck.m`	Validates signal/matrix dimensions (debug utility)
`MLEBus.m` / `mlebusgen.m`	Simulink bus definitions for MLE signals
`MLE.m`	Maximum Likelihood Estimation helper

Archive

Folder	Description
`archive/Simulink_Flight_Simulation/`	Legacy full Simulink PX4 flight sim (pre-thesis)
`Simulink UKF Debugging/`	Scratch Simulink models used during UKF development
`Octo_CantedStacked_FlamingoOLD.mat`	Superseded Flamingo config

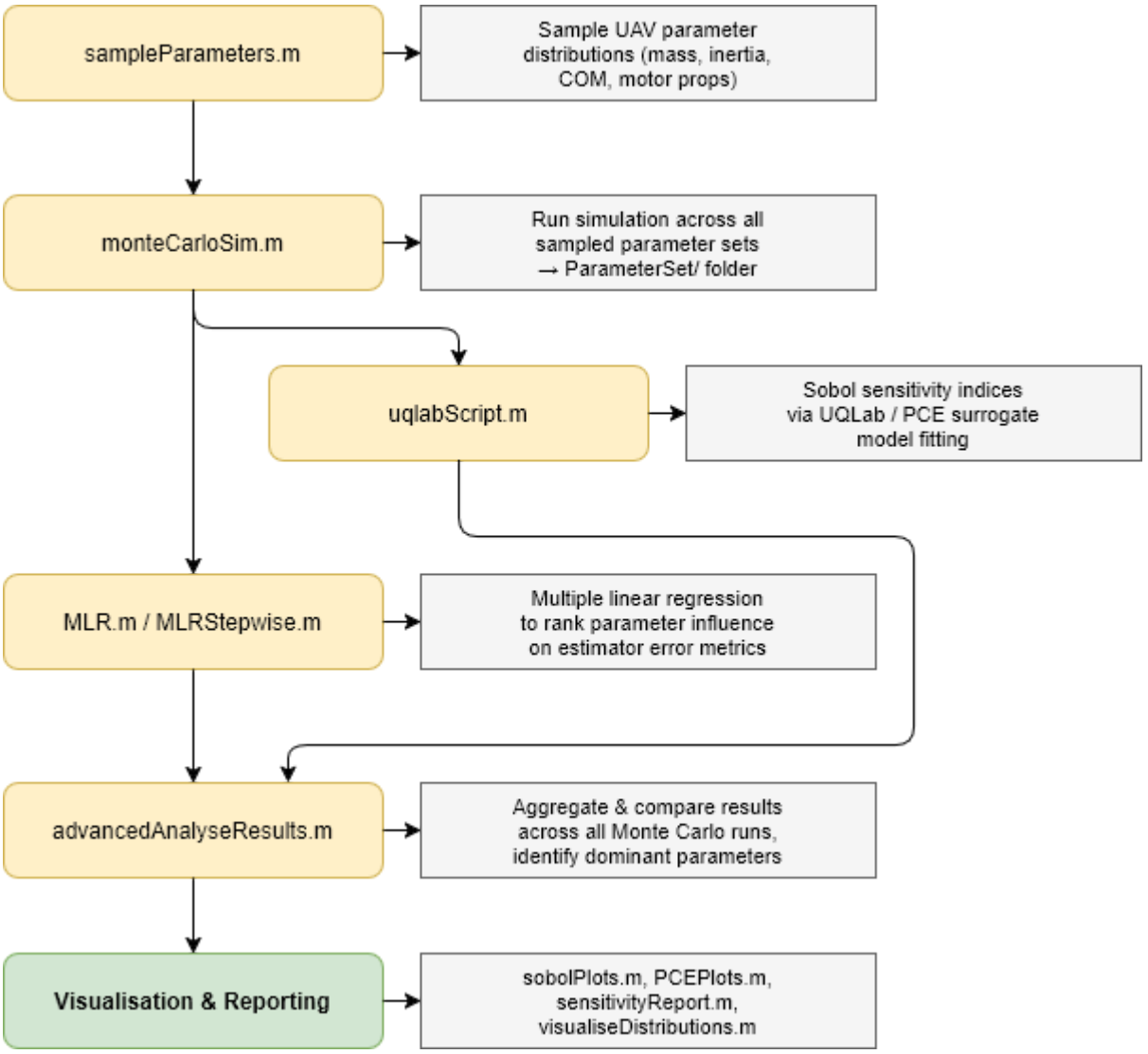
Parameter Sets

Folder	Contents
`ParameterSet/`	50 samples (`ParamSet_001` – `ParamSet_050`) — main Flamingo
`ParameterSetCOMZ/`	Samples varying centre-of-mass Z height
`ParameterSetHEX/`	Hex variant samples
`ParameterSetPHEX/`	PHex variant samples

Section 1 — Sensitivity Analysis

> **Goal:** Determine which UAV physical parameters (inertia, COM, motor properties etc.) have the most significant effect on estimator performance and flight dynamics.
> **Method:** Monte Carlo sampling → simulation → statistical analysis (MLR, Sobol, PCE)
> **Toolbox:** UQLab (MATLAB)

Pipeline Overview



Scripts — `Parameter Sensitivity/`

Core Pipeline

Script	Role	Inputs	Outputs
`sampleParameters.m`	Draws samples from UAV parameter distributions using LHS or random sampling	Parameter bounds defined in `parameters.m`	`ParameterSet/P` files
`monteCarloSim.m`	Runs the estimator simulation across all	`ParameterSet/` folder, simulation config	Per-run result str `.mat` results

Script	Role	Inputs	Outputs
	parameter sets in batch		
`uqlabScript.m`	Fits PCE surrogate model and computes Sobol sensitivity indices via UQLab	Monte Carlo results	Sobol indices, P(
`MLR.m`	Multiple linear regression of parameter values vs output error metrics	Monte Carlo results	Regression coefl
`MLRStepwise.m`	Stepwise MLR — iteratively adds/removes terms to find minimal significant model	Monte Carlo results	Reduced regress
`findSignificantTrends.m`	Identifies parameters with statistically significant influence on outputs	MLR / Sobol results	Ranked paramet
`advancedAnalyseResults.m`	Main batch results aggregation and cross-run comparison	Monte Carlo result files	Summary tables,
`advancedAnalyseResultsSingle.m`	Same as above but for a single selected run — used for deep inspection	Single result `.mat`	Detailed breakdc
`visualiseSingleResult.m`	Plots estimator outputs for	Single result `.mat`	Figures

Script	Role	Inputs	Outputs
	one parameter set run		
`visualiseDistributions.m`	Plots the sampled parameter distributions	`ParameterSet/` files	Histogram / KDE
`sobolPlots.m`	Visualises Sobol first-order and total-order indices	UQLab Sobol output	Bar charts of ser
`PCEPlots.m`	Visualises PCE surrogate model fit and error	UQLab PCE output	Fit quality plots
`sensitivityReport.m`	Compiles key sensitivity results into a printable summary	Analysis outputs	Formatted report
`computeWrench.m`	Computes force/torque wrench from motor commands and UAV geometry	UAV config, motor inputs	Wrench vector `[` `Mz]`

Specialist Analyses

Script	Role
`COMLeakageAnalysis.m`	Analyses how centre-of-mass offset errors propagate through the estimator — "leakage" of COM uncertainty into other parameter estimates
`granularCOMAnalysis.m`	Fine-grained sweep of COM position effects, likely a deeper follow-up to `COMLeakageAnalysis.m`

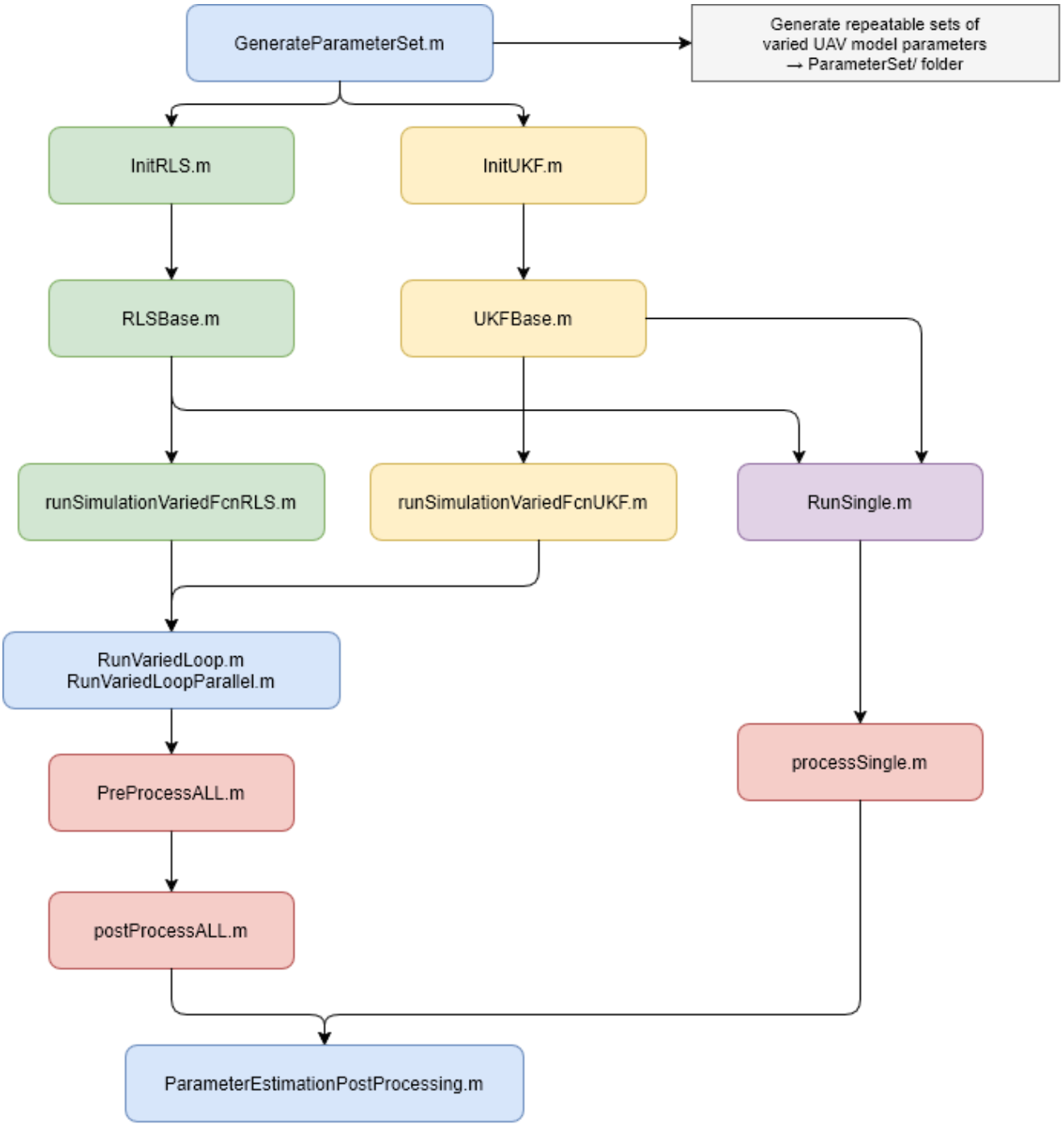
Dependencies

Dependency	Source
Parameter samples	`ParameterSet/`, generated by `GenerateParameterSet.m` (root)
UAV config	`Octo_CantedStacked_Flamingo.mat`
Global parameters	`parameters.m` (root)
UQLab toolbox	External MATLAB toolbox (must be on path)
Simulation runner	`monteCarloSim.m` calls into core sim infrastructure shared with 02_Simulation_RLS_UKF

Section 2 — Simulation: RLS & UKF Parameter Estimation

> **Goal:** Simulate and compare Recursive Least Squares (RLS) and Unscented Kalman Filter (UKF) estimators against known ground-truth parameters using synthetic trajectory data. Runs both single inspectable cases and full Monte Carlo batches.

Pipeline Overview



Scripts

Dataset Setup

Script	Role	Notes
`GenerateParameterSet.m`	Generates and saves the full set of varied UAV model parameter files into `ParameterSet/` — ensures Monte Carlo runs are reproducible across RLS and UKF	Run once before any batch simulation

Initialisation

Script	Estimator	Role
`RLS/InitRLS.m`	RLS	Sets up RLS filter matrices, forgetting factor, initial parameter estimates, signal dimensions
`UKF/InitUKF.m`	UKF	Sets up UKF noise covariances (Q, R), sigma point parameters, initial state/covariance

---

Core Estimator Scripts

Script	Estimator	Role
`RLS/RLSBase.m`	RLS	Core RLS algorithm — regressor construction, recursive weight update, parameter vector evolution
`UKF/UKFBase.m`	UKF	Core UKF algorithm — sigma point propagation, measurement update, state/covariance recursion
`UKF/RVL_UKF.m`	UKF variant	Robust Variable-Lag UKF variant — check if this is the active estimator or a development branch

---

Single Run (Inspection & Debug)

Script	Role
`RunSingle.m`	Runs one simulation with one parameter set through both or either estimator — primary tool for tuning and debugging
`processSingle.m`	Post-processes the output of a single run — plots parameter convergence, error metrics, trajectory tracking

---

Batch / Monte Carlo Runs

Script	Role
`runSimulationVariedFcnRLS.m`	Defines the per-iteration simulation function for RLS — called by the loop runner for each parameter set
`runSimulationVariedFcnUKF.m`	Same as above for UKF

Script	Role
`RunVariedLoop.m`	Iterates over all parameter sets in `ParameterSet/` sequentially, calling the varied function
`RunVariedLoopParallel.m`	Same as `RunVariedLoop.m` but uses `parfor` for parallel execution — use for large batches

---

Pre/Post Processing (Monte Carlo)

Script	Role
`DataProcessing/PreProcessALL.m`	Loads and prepares all batch results before analysis — aligns data, checks for failed runs
`DataProcessing/preProcessRLS.m`	RLS-specific preprocessing — extracts RLS parameter traces from batch results
`DataProcessing/preProcessUKF.m`	UKF-specific preprocessing — extracts UKF state/covariance traces from batch results
`DataProcessing/postProcessALL.m`	Main batch post-processing — computes error metrics across all runs, prepares data for sensitivity analysis
`DataProcessing/postProcessRLS.m`	RLS-specific post-processing — convergence rates, final parameter error
`DataProcessing/postProcessUKF.m`	UKF-specific post-processing — same metrics for UKF
`ParameterEstimationPostProcessing.m`	Top-level aggregation — combines RLS and UKF processed results for direct comparison

---

Hexacopter Variants (`HEXTest/`)

Mirror of the main RLS/UKF pipeline configured for the hexacopter platform.

Script	Equivalent
`HEXTest/InitRLSHEX.m`	`RLS/InitRLS.m`
`HEXTest/InitUKFHEX.m`	`UKF/InitUKF.m`
`HEXTest/RLSBaseHEX.m`	`RLS/RLSBase.m`
`HEXTest/UKFBaseHEX.m`	`UKF/UKFBase.m`
`HEXTest/runSimulationVariedFcnRLSHEX.m`	`RLS/runSimulationVariedFcnRLS.m`
`HEXTest/runSimulationVariedFcnUKFHEX.m`	`UKF/runSimulationVariedFcnUKF.m`

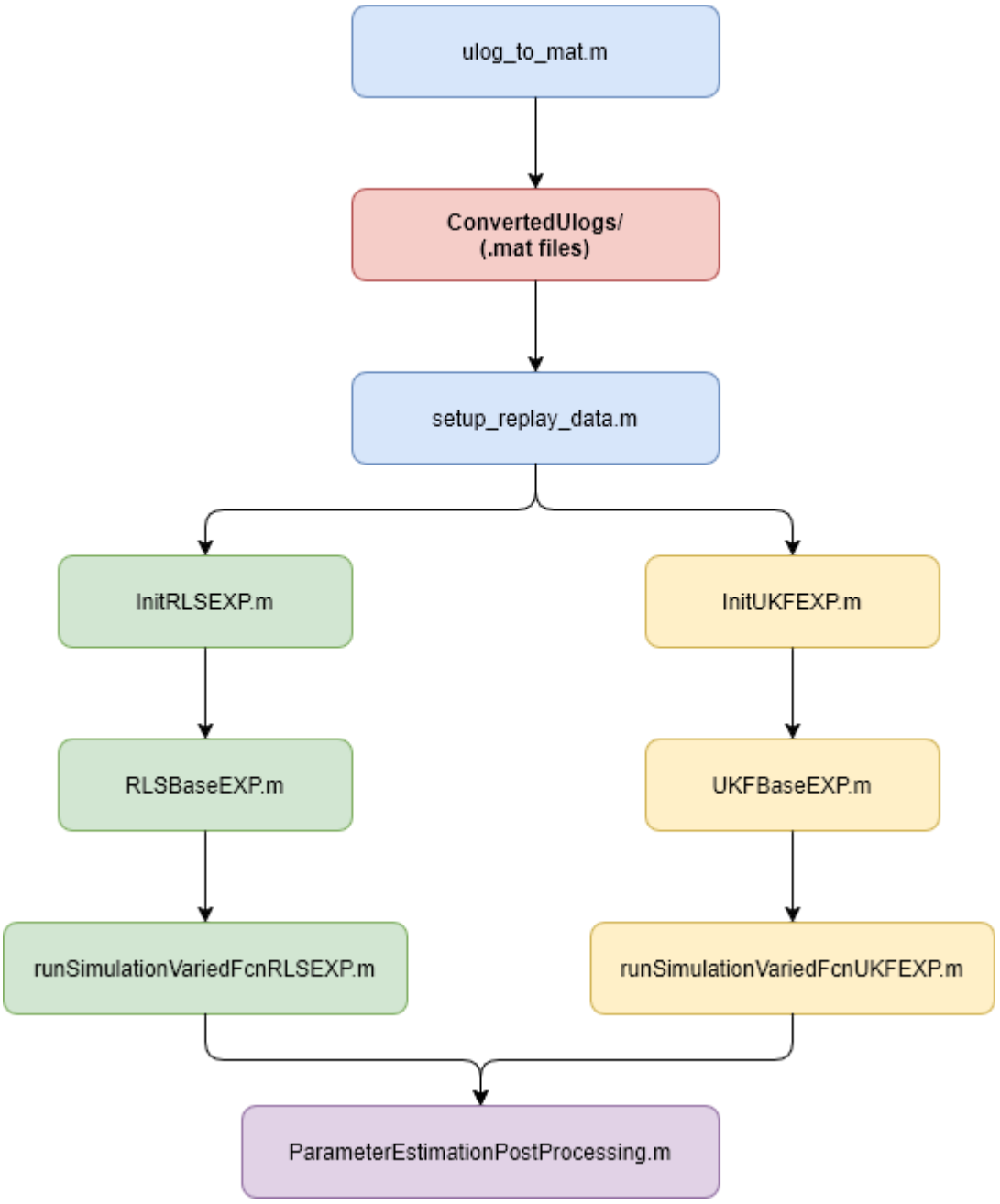
Dependencies

Dependency	Source
UAV config	`Octo_CantedStacked_Flamingo.mat`, `Hex.mat`
Parameter sets	`ParameterSet/` — generated by `GenerateParameterSet.m`
Trajectories	`generate_trajectory.m`, `generate_6dof_step_trajectory.m`
Global parameters	`parameters.m`
Performance scoring	`evaluateUAVPerformance.m`
Results feed into	01_Sensitivity_Analysis (Monte Carlo outputs) and 04_Mixer_Testing (estimated parameters)

Section 3 — Real World Data Processing

> **Goal:** Replay RLS and UKF estimators on real flight data from the Flamingo UAV, compare estimated parameters against each other and against known physical values. Mirrors the Section 2 simulation pipeline but driven by logged flight data rather than synthetic trajectories.

Pipeline Overview



Stage 1 — Data Conversion

Script	Role	Inputs	Outputs
`EXP_Processing/ulog_to_mat.m`	Converts raw PX4 `.ulg` flight logs into MATLAB `.mat` format, extracting all relevant topics (actuator outputs, IMU, attitude, position)	Raw `.ulg` from `Flamingo Data/`	`.mat` files in `ConvertedUlogs/`

Stage 2 — Data Setup & Inspection

Script	Role	Notes
`EXP_Processing/setup_replay_data.m`	Loads a converted log and packages it into the data structure expected by the estimator replay scripts — the EXP equivalent of `GenerateParameterSet.m`	Run before Init scrip
`EXP_Processing/interpolate.m`	Resamples/interpolates flight log signals onto a uniform time grid — needed because PX4 topics log at different rates	Utility called by `setup_replay_data`
`EXP_Processing/plot_flight_data.m`	Plots raw converted flight data for sanity checking — attitudes, rates, motor commands, position	Inspection tool

---

Stage 3 — Initialisation

Mirrors `RLS/InitRLS.m` and `UKF/InitUKF.m` from Section 2 but tuned for real data characteristics (noise levels, sample rates from actual hardware).

Script	Estimator	Role
`EXP_Processing/InitRLSEXP.m`	RLS	Initialises RLS filter for real data replay — forgetting factor, initial params, regressor config tuned to flight log sample rate
`EXP_Processing/InitUKFEXP.m`	UKF	Initialises UKF for real data replay — Q/R covariances tuned to measured sensor noise

---

Stage 4 — Estimator Replay

Script	Estimator	Role
`EXP_Processing/RLSBaseEXP.m`	RLS	Core RLS algorithm driven by real flight

Script	Estimator	Role
		data signals instead of simulation outputs
`EXP_Processing/UKFBaseEXP.m`	UKF	Core UKF driven by real flight data
`EXP_Processing/runSimulationVariedFcnRLSEXP.m`	RLS	Per-iteration function for batch replay of RLS across multiple flight logs
`EXP_Processing/runSimulationVariedFcnUKFEXP.m`	UKF	Same for UKF

Stage 5 — Post Processing

Shared with Section 2 — real world results feed into the same post-processing scripts for direct comparison with simulation results.

Script	Role
`ParameterEstimationPostProcessing.m`	Aggregates EXP replay results alongside simulation results for cross-comparison

---

Flight Datasets — `Flamingo Data/`

Dataset Overview

Dataset	Folder	Logs	Config	Notes
Normal baseline flight	`Normal Flight/`	3 × `.ulg`	Standard Flamingo	Unexcited hover/flight for baseline
RLS excitation tuning	`RLS/`	4 logs at amplitudes 0.01, 0.05, 0.075, 0.1	Standard Flamingo	Used to select optimal excitation amplitude
RLS 0.075 samples	`RLS/0.075 Samples/`	5 logs	Standard Flamingo	Full sample set at chosen excitation level
RLS heavy payload (low excitation)	`RLS Heavy Low/`	5 flights (L1–L5)	Flamingo + payload	Tests estimator under changed inertial properties

Dataset	Folder	Logs	Config	Notes
RLS low larger excitation	`RLS Low Larger Excitation/`	1 log	Flamingo	Single test with increased excitation
RLS 5Ah battery (1380g)	`RLS Sample 5Ah weight 1380g/`	5 samples (mocap + ulog)	Flamingo 5Ah	Primary dataset — mocap ground truth available

Motion Capture Data

Each mocap session produces four files per trial:

Extension	Contents
`.csv`	Exported rigid body position/orientation timeseries
`.x2d`	Raw Qualisys 2D camera data
`.xcp`	Qualisys camera calibration
`.system`	Qualisys system/session config

> **Ground truth use:** The `.csv` mocap data provides position and attitude ground truth for validating estimator outputs against known physical motion.

Dependencies

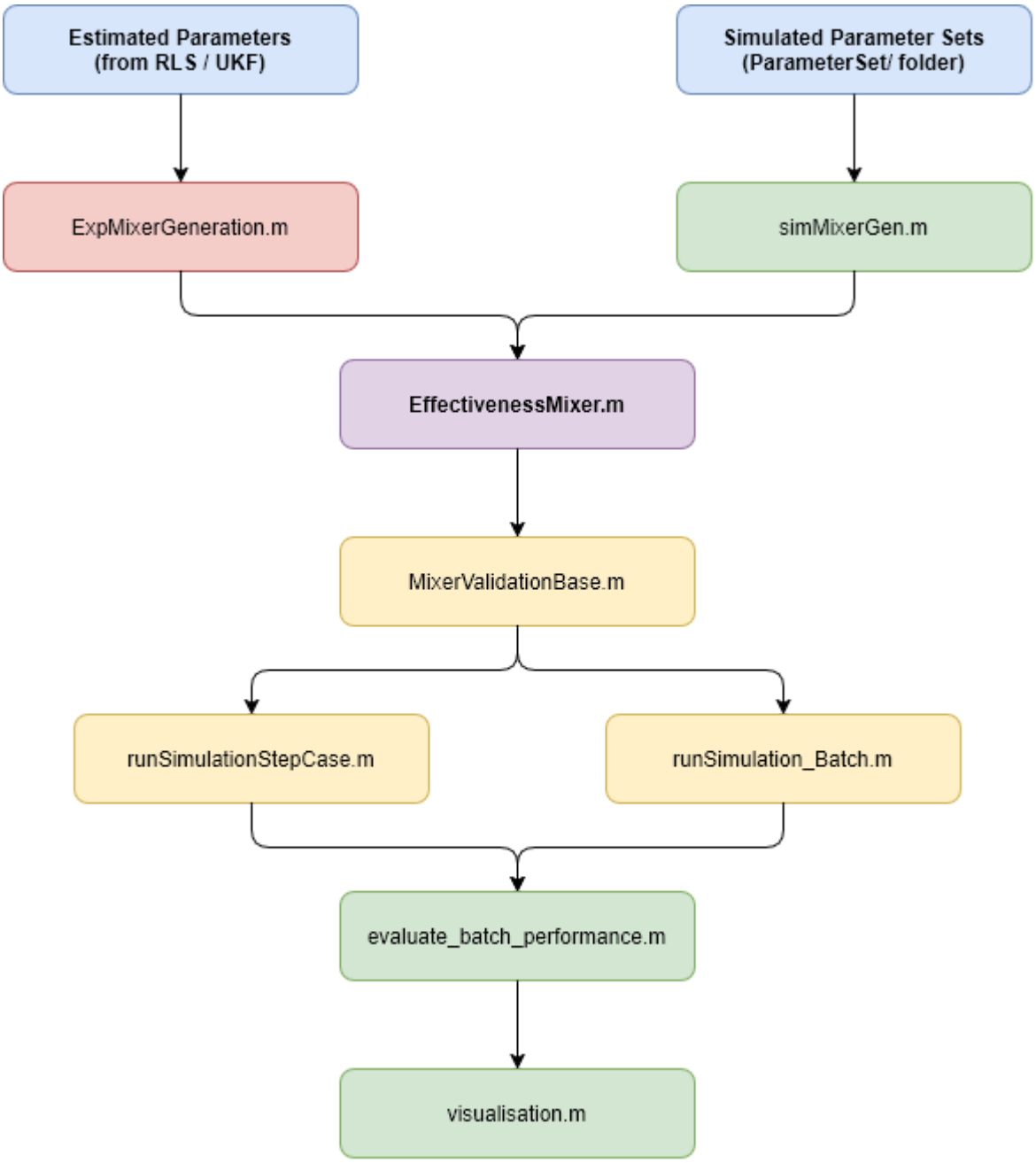
Dependency	Source
Raw flight logs	`Flamingo Data/` subfolders
UAV config	`Octo_CantedStacked_Flamingo.mat`
Global parameters	`parameters.m`
Time sync utility	`EXP_Processing/interpolate.m`
Results feed into	`ParameterEstimationPostProcessing.m` and 04_Mixer_Testing

Section 4 — Mixer Calculation & Testing

> **Goal:** Use estimated UAV parameters (inertia, geometry, motor properties) to compute an updated control effectiveness mixer, then validate the new mixer's flight performance against the nominal/default mixer via batch simulation.
>
> **Background:** The PX4 control effectiveness mixer maps desired forces/torques to individual motor commands. If the UAV's true physical parameters differ from

> the nominal values baked into the default mixer, performance degrades. This section tests whether mixers derived from estimated parameters improve control.

Pipeline Overview



Stage 1 — Mixer Generation

Two parallel paths generate mixers — one from real estimated parameters, one from simulated parameter sets — both feeding into the same `EffectivenessMixer.m` core calculation.

Script	Source Data	Role
`DataProcessing/ExpMixerGeneration.m`	Estimated parameters from real flight EXP replay (Section 3)	Packages estimated inertia, geometry and motor parameters into the format required by `EffectivenessMixer.m`
`DataProcessing/simMixerGen.m`	Parameter sets from `ParameterSet/` folder (Section 2)	Same packaging but from simulated/sampled parameter sets — used for batch mixer validation
`EffectivenessMixer.m`	Output of either above	Core mixer calculation — constructs the control effectiveness matrix from UAV geometry and motor properties, computes the pseudoinverse mixer

---

Stage 2 — Validation Setup

Script	Role	Notes
`MixerTesting/MixerValidationBase.m`	Sets up the validation simulation environment — loads a mixer, configures the UAV model, defines the test trajectory or step inputs	Central setup script, analogous to `RLSBase.m` / `UKFBase.m` in Section 2

Stage 3 — Simulation Runs

Two modes of validation — single inspectable step response and full batch:

Script	Role	When to Use
`MixerTesting/runSimulationStepCase.m`	Runs a single step input case with a given mixer — plots attitude/position response for visual inspection	Tuning and debugging a specific mixer
`MixerTesting/runSimulation_Batch.m`	Runs the validation simulation across	Comparing estimated vs

Script	Role	When to Use
	multiple mixers and/or parameter sets in batch	nominal mixer across many conditions

---

Stage 4 — Evaluation & Visualisation

Script	Role
`MixerTesting/evaluate_batch_performance.m`	Computes performance metrics across all batch validation runs — tracking error, settling time, overshoot. Compares estimated-parameter mixer against nominal mixer
`MixerTesting/visualisation.m`	Plots batch evaluation results — performance metric distributions, estimated vs nominal comparison figures

---

Dependencies

Dependency	Source
Estimated parameters (EXP)	Output of 03_RealWorld_Processing via `ExpMixerGeneration.m`
Simulated parameter sets	`ParameterSet/` folder, output of 02_Simulation_RLS_UKF
UAV geometry & motor config	`Octo_CantedStacked_Flamingo.mat`
Global parameters	`parameters.m`
Step trajectory inputs	`generate_6dof_step_trajectory.m`
Performance scoring	`evaluateUAVPerformance.m`